

Computerarchitektur - Probeklausur

Lösungen

Aufgabe 1 - Leistungsbewertung

a) Beschleunigung des Multiplikationsteils:

$$S_{\text{Mul}} = 180 / 6 = 30$$

Amdahls Gesetz:

$$S_{\text{gesamt}} = 1 / (\text{Anteil}_{\text{unbenutzt}} + \text{Anteil}_{\text{benutzt}} / S_{\text{Mul}})$$

$$S_{\text{gesamt}} = 1 / (0,30 + 0,70 / 30) = 3,0928$$

Ergebnis: Die Gesamtbeschleunigung beträgt ungefähr 3,09.

b) Gesucht ist die Beschleunigung S_{Mul} des Multiplikationsteils:

$$2,5 = 1 / (0,20 + 0,80 / S_{\text{Mul}})$$

$$0,40 = 0,20 + 0,80 / S_{\text{Mul}} \Rightarrow S_{\text{Mul}} = 4$$

$$\text{Taktzyklen} = 180 / 4 = 45$$

Ergebnis: Die Multiplikation muss um den Faktor 4 beschleunigt werden und benötigt danach 45 Taktzyklen.

Aufgabe 2 - Fehlertoleranz

a) Prüf-Bit 1 prüft die Positionen 1, 3, 5, 7, 9 und 11; Prüf-Bit 2 die Positionen 2, 3, 6, 7, 10 und 11; Prüf-Bit 4 die Positionen 4, 5, 6, 7 und 12; Prüf-Bit 8 die Positionen 8, 9, 10, 11 und 12. Bei ungerader Parität muss jede Prüfgruppe insgesamt eine ungerade Anzahl von Einsen besitzen.

Wort-Nr.	1	2	3	4	5	6	7	8	9	10	11	12
1	0	0	1	1	0	1	1	1	0	1	1	0
2	1	0	0	0	1	1	0	1	1	0	0	1
3	1	1	1	1	1	0	0	1	0	1	0	1
4	1	0	0	1	0	1	1	1	1	1	0	0
5	1	0	1	1	0	0	1	0	1	0	1	1

b) Für 58 Daten-Bits und r Prüf-Bits gilt:

$$2^r \geq 58 + r + 1$$

$$\text{Für } r = 6: 2^6 = 64 < 58 + 6 + 1 = 65$$

$$\text{Für } r = 7: 2^7 = 128 \geq 58 + 7 + 1 = 66$$

Ergebnis: Es werden 7 Prüf-Bits benötigt.

Aufgabe 3 - Speicherhierarchie

1) Die parallelen Zugriffe auf L1 und L2 werden mit der jeweiligen Trefferzeit gewichtet; L3 und Hauptspeicher werden danach sequentiell berücksichtigt.

Fall	Wahrscheinlichkeit	Zeit
L1-Hit	0,75	2 ns
L1-Miss, L2-Hit	$0,25 * 0,80 = 0,20$	6 ns
L1-/L2-Miss, L3-Hit	$0,25 * 0,20 * 0,75 = 0,0375$	$6 + 24 = 30$ ns
L1-/L2-/L3-Miss	$0,25 * 0,20 * 0,25 = 0,0125$	$6 + 24 + 90 = 120$ ns

$$A = 0,75 * 2 + 0,20 * 6 + 0,0375 * 30 + 0,0125 * 120 = 5,325 \text{ ns}$$

2) Ohne L3 greifen alle L1-/L2-Misses direkt auf den neuen Hauptspeicher mit Zugriffszeit x zu:

$$A_{\text{neu}} = 0,75 * 2 + 0,20 * 6 + 0,05 * (6 + x)$$

$$A_{\text{neu}} = 3 + 0,05x$$

$$3 + 0,05x = 5,325 \Rightarrow x = 46,5 \text{ ns}$$

Ergebnis: Der neue Hauptspeicher darf höchstens 46,5 ns benötigen. Für eine strikt schnellere mittlere Zugriffszeit muss $x < 46,5$ ns gelten.

Aufgabe 4 - Adressierung

Direct lädt den Inhalt der angegebenen Speicherzelle; Indirect benutzt deren Inhalt als endgültige Adresse; Indexed addiert Offset und Register; Based-Indexed addiert zwei Register.

Lesezugriff mit Modus der Adressierung	Wert im Akkumulatorregister
Load Direct 60	140
Load Indirect 220	35
Load Immediate 200	200
Load Indirect-Register-Mode R3	35
Load Register-Mode R2	160
Load Indirect-Register-Mode R4	125
Load Based-Indexed-Mode R1 R2	215
Load Indexed-Mode 80 R1	55
Load Direct 180	35
Load Indirect 240	390

Register

Register	Wert des Registers
R1	300
R2	160
R3	180
R4	420

Speicher

Speicheradresse	Speicherinhalt
60	140
140	390
180	35
220	180
240	140
380	55
420	125
460	215

Begründung: R3 = 180, weil Speicher[180] = 35. R4 = 420, weil Speicher[420] = 125. Aus R1 + R2 = 460 und R2 = 160 folgt R1 = 300. Damit zeigt 80 + R1 auf Adresse 380.

Aufgabe 5 - Instruktionsabhängigkeit und Pipelining

a1) Bei fester Reihenfolge darf keine spätere Instruktion an einer blockierten früheren Instruktion vorbeiziehen.

Minimale Ausführungszeit: 6 Taktzyklen.

Taktzyklus	AE1	AE2	AE3
1	I1	I2	I3
2	I4		
3	I5		
4	I6	I7	I8
5	I9	I10	I11
6	I12		
7			
8			
9			
10			
11			
12			

Aufgabe 5 - Fortsetzung

a2) Bei modifizierbarer Reihenfolge können I7 und I8 unmittelbar nach ihren Vorgängern I2 beziehungsweise I3 ausgeführt werden. Die unabhängigen Instruktionen I9 bis I12 können freie Ausführungseinheiten frühzeitig belegen.

Minimale Ausführungszeit: 4 Taktzyklen.

Taktzyklus	AE1	AE2	AE3
1	I1	I2	I3
2	I4	I7	I8
3	I5	I9	I10
4	I6	I11	I12
5			
6			
7			
8			
9			
10			
11			
12			

a3) Auch mit 14 AE werden mindestens 4 Takte benötigt. Die RAW-Kette I1 -> I4 -> I5 -> I6 besitzt vier aufeinanderfolgende Stufen. Mehr Ausführungseinheiten können diese Abhängigkeit nicht verkürzen. Eine mögliche Belegung ist:

Taktzyklus	Ausgeführte Instruktionen
1	I1, I2, I3, I9, I10, I11, I12
2	I4, I7, I8
3	I5
4	I6

b) Vollständige Abhängigkeiten des Codes:

RAW: I1 -> I3 (P11); I2 -> I4 (P12); I3 -> I4 (P13); I4 -> I5 (P14); I4 -> I6 (P14); I4 -> I7 (P14); I4 -> I8 (P14)

WAR: I1 -> I5 (P21); I1 -> I6 (P31); I3 -> I7 (P11); I4 -> I8 (P12)

WAW: I1 -> I7 (P11); I2 -> I8 (P12)

Die letzten Abhängigkeiten werden erst durch die Instruktionen I7 und I8 sichtbar: I3 -> I7 (WAR), I1 -> I7 (WAW), I4 -> I8 (WAR) und I2 -> I8 (WAW).

Aufgabe 6 - Mikroarchitektur - Analyse eines ISA-IJVM-Programms

1) Zuerst werden die dezimal angegebenen Wort-Adressen den Hex-Operanden zugeordnet: 13 = 0x0D = A, 21 = 0x15 = B, 37 = 0x25 = C, 52 = 0x34 = X und 59 = 0x3B = Y.

Start-Byte	Bytefolge	IJVM-Mnemonic
1	0x15 0x0D	ILOAD A
3	0x15 0x15	ILOAD B
5	0x9F 0x00 0x0D	IF_ICMPEQ 18
8	0x15 0x0D	ILOAD A
10	0x59	DUP
11	0x15 0x25	ILOAD C
13	0x60	IADD
14	0x60	IADD
15	0xA7 0x00 0x08	GOTO 23
18	0x15 0x15	ILOAD B
20	0x10 0x07	BIPUSH 7
22	0x64	ISUB
23	0x36 0x34	ISTORE X
25	0x15 0x34	ILOAD X
27	0x10 0x02	BIPUSH 2
29	0x60	IADD
30	0x36 0x3B	ISTORE Y

Sprungberechnung: Der Offset wird auf die Byte-Adresse des Opcodes addiert. Bei Byte 5 gilt $5 + 0x000D = 18$. Bei Byte 15 gilt $15 + 0x0008 = 23$.

2a) $A = 9$, $B = 4$, $C = 6$: A und B sind ungleich, daher wird IF_ICMPEQ nicht ausgeführt. Der linke Pfad berechnet $X = A + A + C = 24$. Danach gilt $Y = X + 2 = 26$.

2b) $A = 4$, $B = 4$, $C = 6$: A und B sind gleich, daher wird zu Byte 18 gesprungen. Der rechte Pfad berechnet $X = B - 7 = -3$. Danach gilt $Y = X + 2 = -1$.

3) Stapelhöhen:

Pfad	Stapelentwicklung (unten ... oben)	Maximum
$A \neq B$	nach Byte 4: A, B; nach Byte 7: leer; nach Byte 9: A; nach Byte 10: A, A; nach Byte 12: A, A, C; nach Byte 13: A, A + C; nach Byte 14: $2A + C$	3 nach Byte 12
$A = B$	nach Byte 4: A, B; nach Byte 7: leer; nach Byte 19: B; nach Byte 21: B, 7; nach Byte 22: $B - 7$	2 nach Byte 4 oder 21
gemeinsam	nach Byte 26: X; nach Byte 28: X, 2; nach Byte 29: $X + 2$; nach Byte 31: leer	2 nach Byte 28

Gesamtmaximum: 3 Wörter, erreicht nach Byte 12 (nach vollständiger Ausführung von ILOAD C).

Aufgabe 7 - Sekundärspeicher - Magnetische Festplatte

a) Die 4 Scheiben besitzen zusammen 8 Oberflächen und damit 8 Spuren pro Zylinder.

Außenbereich = $8192 * 8 * 1024 * 512 \text{ Byte} = 34\,359\,738\,368 \text{ Byte} = 32 \text{ GiB}$

Innenbereich = $8192 * 8 * 512 * 512 \text{ Byte} = 17\,179\,869\,184 \text{ Byte} = 16 \text{ GiB}$

Gesamt = $51\,539\,607\,552 \text{ Byte} = 51,54 \text{ GByte} = 48 \text{ GiB}$

b) 2 MiB entsprechen 4096 Sektoren. Auf einer äußeren Spur liegen 1024 Sektoren, daher werden 4 Spuren benötigt.

$6000 \text{ UpM} = 100 \text{ Ups} \Rightarrow T = 1 / 100 \text{ s} = 10 \text{ ms}$

Mittlerer Rotational Delay = 5 ms

Transfer Time für 4 vollständige Spuren = $4 * 10 \text{ ms} = 40 \text{ ms}$

$t = 4 \text{ ms} + 4 * 5 \text{ ms} + 3 * 0,5 \text{ ms} + 40 \text{ ms} = 65,5 \text{ ms}$

c) Bei 4 Sektoren pro Spur werden $4096 / 4 = 1024$ Spuren verwendet. Es treten 1023 Spurwechsel auf. Die reine Transfer Time bleibt 40 ms.

$t = 4 \text{ ms} + 1024 * 5 \text{ ms} + 1023 * 0,5 \text{ ms} + 40 \text{ ms} = 5675,5 \text{ ms}$

Ergebnisse: Kapazität = 51,54 GByte (48 GiB), zusammenhängend = 65,5 ms, verteilt = 5675,5 ms.

Aufgabe 8 - Cache-Kohärenz mit MOESI

a) Bewertung der Zustandskombinationen:

Nr.	Zustände C1/C2/C3/CM	R/F	Begründung
1	E/I/I/V	R	Eine einzige saubere Cache-Kopie; Hauptspeicher gültig.
2	S/S/I/V	R	Mehrere saubere geteilte Kopien; Hauptspeicher gültig.
3	O/S/I/I	R	Owner besitzt die aktuelle Kopie; weitere Shared-Kopie; CM ungültig.
4	M/I/I/I	R	Genau eine modifizierte Kopie; CM ungültig.
5	M/S/I/I	F	M darf nicht gleichzeitig mit einer weiteren gültigen Cache-Kopie vorkommen.
6	I/O/S/V	F	Bei O ist der Hauptspeicher nicht aktuell und muss I sein.
7	S/I/I/V	F	Eine einzige saubere Cache-Kopie wird als E und nicht als S geführt.

b) Zustände nach den Operationen:

Schritt	Operationen	CLA: C1	CLA: C2	CLA: C3	CLA: CM	CLC: C1	CLC: C2	CLC: C3	CLC: CM
vor Beginn		I	I	M	I	E	I	I	V
1	P2: read VA	I	S	O	I	E	I	I	V
gehört zu Takt 1	P3: write VC	I	S	O	I	I	I	M	I
2	P1: write VA	M	I	I	I	I	I	M	I
gehört zu Takt 2	P3: entferne CLC	M	I	I	I	I	I	I	V
3	P2: read VA	O	S	I	I	I	I	I	V
gehört zu Takt 3	P1: read VC	O	S	I	I	E	I	I	V

Takt 1: Beim Read von VA liefert C3 die modifizierte Zeile, wird O und C2 erhält S. Der gleichzeitige Write auf die andere Zeile CLC macht C3 dort zu M und invalidiert C1. Takt 2: P1 übernimmt CLA exklusiv zum Schreiben und wird M; beim Entfernen der modifizierten CLC-Zeile wird in den Hauptspeicher zurückgeschrieben. Takt 3: Der Read von VA erzeugt O/S. Der gleichzeitige Read von VC betrifft die andere Zeile und erzeugt dort E.